



Curso Javascript básico

Aula 08 - Trabalhando com Strings



O que é uma String?

Por definição, strings são sequências de caracteres alfanuméricos (letras, números e/ou símbolos) amplamente usadas em programação. Em Javascript, uma string sempre estará entre aspas.

```
var frase = "Mergulhando em tecnologia Javascript";  
var frase = 'Mergulhando em tecnologia Javascript';
```



O que é uma String?

Mas eu declaro minhas strings com aspas duplas ou simples?

Podemos colocar nossas strings entre aspas duplas ou simples. Para o JavaScript, não há diferença, já que ele considera as duas formas de declaração válidas.

É importante ressaltar que, depois que a sequência de caracteres for definida, a string é imutável, ou seja, não poderá ter seu valor alterado. Então, como manipular a string?

Sempre que manipulamos uma string, é criada uma nova instância dela por baixo dos panos, o que significa que será gerado um novo espaço na memória com uma cópia do valor da string. Por isso, temos que utilizar uma variável para armazená-la.



Objeto String

A linguagem JavaScript traz ainda como recurso um objeto global String que nos permite criar ou converter um tipo em uma string.

```
var numero = 256 ;  
var convertidoEmString = new String(numero) ;
```



Objeto String

A linguagem JavaScript traz ainda como recurso um objeto global String que nos permite criar ou converter um tipo em uma string.

```
var numero = 256 ;  
var convertidoEmString = new String(numero) ;
```

Também é possível converter outros tipos primitivos (por exemplo, números e booleanos) em strings com o método toString().

```
console.log(num.toString())
```



Usando Strings

É possível interpolar, concatenar, checar posições de caracteres ou ainda substituir partes de strings.



Concatenando strings

Quando falamos em concatenar strings, quer dizer que vamos juntar duas ou mais strings e formar uma nova.

```
var nome = "André"  
var sobreNome = "Silva"  
var nomeCompleto = "Meu nome completo é : " + nome + sobreNome
```



Interpolando strings (template strings)

A interpolação de strings é um recurso bem interessante, presente em diversas linguagens. No JavaScript, é uma alternativa mais prática para manipular string sem a necessidade de fazer concatenação, porque para textos maiores, concatenar pode ser um pouco trabalhoso.

Usando as chamadas template strings ou templates literais, a pessoa desenvolvedora consegue ter uma flexibilidade maior no trabalho com strings, além de facilitar a escrita e leitura do código.

```
var nome = "André"  
var saudacoes = `Seja bem-vindo ${nome}`
```




Caractere de Fuga

Como as strings devem ser escritas entre aspas, o JavaScript não entenderá essa string.

```
var text = "We are the so-called \"Vikings\" from the north.\";
```



Caractere de Fuga

Seis outras sequências de escape são válidas em JavaScript:

<i>Código</i>	<i>Resultado</i>
<i>\b</i>	<i>Backspace</i>
<i>\f</i>	<i>Form Feed</i>
<i>\n</i>	<i>Nova Linha</i>
<i>\r</i>	<i>Carriage Return</i>
<i>\t</i>	<i>Tabulação Horizontal</i>
<i>\v</i>	<i>Tabulação Vertical</i>



Métodos para strings

O JavaScript diferencia strings como tipos primitivos (com aspas duplas ou simples) de objetos Strings (quando usamos a palavra reservada `new`). Mas, por baixo dos panos toda string, mesmo as que criamos com a chamada “forma literal”, por exemplo `const texto = “valor texto”`, acaba convertida para um objeto do tipo `String`. Por isso, temos acesso a uma série de métodos e propriedades deste objeto.



Métodos

Métodos	Descrição
<code>charAt()</code>	Retorna o caractere em um índice especificado (posição)
<code>charCodeAt()</code>	Retorna o Unicode do caractere em um índice especificado
<code>concat()</code>	Retorna duas ou mais strings unidas
<code>endsWith()</code>	Retorna se uma string termina com um valor especificado
<code>fromCharCode()</code>	Retorna valores Unicode como caracteres
<code>includes()</code>	Retorna se uma string contém um valor especificado
<code>indexOf()</code>	Retorna o índice (posição) da primeira ocorrência de um valor em uma string
<code>lastIndexOf()</code>	Retorna o índice (posição) da última ocorrência de um valor em uma string
<code>localeCompare()</code>	Compara duas strings na localidade atual
<code>match()</code>	Pesquisa uma string por um valor ou uma expressão regular e retorna as correspondências
<code>repeat()</code>	Retorna uma nova string com um número de cópias de uma string
<code>replace()</code>	Pesquisa uma string por um valor ou uma expressão regular e retorna uma string onde os valores são substituídos



Métodos

Métodos	Descrição
<code>search()</code>	Pesquisa um valor ou expressão regular em uma string e retorna o índice (posição) da correspondência
<code>slice()</code>	Extraí uma parte de uma string e retorna uma nova string
<code>split()</code>	Divide uma string em uma matriz de substrings
<code>startsWith()</code>	Verifica se uma string começa com caracteres especificados
<code>substr()</code>	Extraí um número de caracteres de uma string, de um índice inicial (posição)
<code>substring()</code>	Extraí caracteres de uma string, entre dois índices especificados (posições)
<code>toLocaleLowerCase()</code>	Retorna uma string convertida em letras minúsculas, usando a localidade do host
<code>toLocaleUpperCase()</code>	Retorna uma string convertida em letras maiúsculas, usando a localidade do host
<code>toLowerCase()</code>	Retorna uma string convertida em letras minúsculas
<code>toString()</code>	Retorna uma string ou um objeto string como uma string
<code>toUpperCase()</code>	Retorna uma string convertida em letras maiúsculas
<code>trim()</code>	Retorna uma string com espaços em branco removidos
<code>valueOf()</code>	Retorna o valor primitivo de uma string ou um objeto string